

Loss Functions, Worked Out by Hand

MSE, Huber, BCE-with-logits and Cross-Entropy, computed end to end

A loss function is a single number that scores how wrong a prediction is, and — just as important — the *gradient* of that number is what the optimiser actually consumes. This document takes the four losses the course uses in its training loop and computes each one, plus its gradient, on tiny hand-checkable inputs. Nothing is hand-waved: every value below was produced by the NumPy/PyTorch program in Appendix A and matches `torch.nn` to three decimals.

What this document does. It fixes one small input for each loss, then walks the arithmetic from raw inputs to the scalar loss and on to $\partial L / \partial(\text{input})$ — the quantity `loss.backward()` writes into your tensors. The regression pair, the binary logit, and the three-class logit vector are all small enough to verify with a calculator.

How to read this. Each loss gets its own section with a worked numerical derivation and a *Mini-example* or coloured box isolating the one idea that trips people up. Read it to understand *why* `CrossEntropyLoss` wants raw logits, why `BCEWithLogitsLoss` is numerically safer than sigmoid-then-BCE, and why the cross-entropy gradient collapses to the beautifully simple softmax — onehot.

Concepts covered, in order: what a loss is (forward score + backward gradient) · MSE and its $2e/n$ gradient · Huber / Smooth-L1, quadratic core and linear tail · outlier robustness, MSE vs Huber · the sigmoid and binary cross-entropy · the numerically stable BCE-with-logits form · softmax, log-softmax and cross-entropy · the softmax — onehot gradient · a decision table for picking a loss.

Internal learning material. Numbers are reproducible from the appendix code.

v1.0

1 The setup: a loss is two numbers in one

Training a model is a search. At each step the model emits a prediction, a **loss function** L collapses prediction-versus-truth into a single scalar, and the optimiser nudges the parameters *down* the slope of that scalar. So a loss is really two things at once:

Direction	Quantity	Who consumes it
forward	the scalar L (a score)	you, for logging / early stopping
backward	the gradient $\partial L / \partial(\text{pred})$	the optimiser, to update weights

We work four losses end to end. Each section computes the scalar *and* the gradient, because the gradient is the part the training loop actually uses.

Intuition

Picture each loss as a bowl plotted against the residual $e = \hat{y} - y$ (regression) or against the logit (classification). **MSE** is a steep parabola — far-off points are punished by the *square* of their error. **Huber** is a parabola in the middle that flattens into straight lines in the tails, so one wild outlier cannot dominate. **Cross-entropy** rewards putting probability mass on the true class and shoots to $+\infty$ as that probability heads to 0. The *shape* of the bowl is the loss’s personality; the *slope* of the bowl is its gradient.

2 Step 1 — Mean Squared Error

Definition. For predictions $\hat{y}$ and targets y with n entries,

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad \frac{\partial L_{\text{MSE}}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i).$$

Worked example (the course’s regression pair). $\hat{y} = [2.5, 0.0, 2.1, 7.8]$, $y = [3.0, -0.5, 2.0, 7.0]$, so $n = 4$. First the residuals $e_i = \hat{y}_i - y_i$, then their squares:

$$e = [-0.5, 0.5, 0.1, 0.8], \quad e^2 = [0.25, 0.25, 0.01, 0.64].$$

$$L_{\text{MSE}} = \frac{0.25 + 0.25 + 0.01 + 0.64}{4} = \frac{1.15}{4} = \boxed{0.287}.$$

The gradient is just the residuals doubled and averaged:

$$\frac{\partial L}{\partial \hat{y}} = \frac{2}{4} e = 0.5 \cdot [-0.5, 0.5, 0.1, 0.8] = [-0.25, 0.25, 0.05, 0.4].$$

Notice the gradient grows *linearly* with the residual: the worse the prediction, the harder MSE pulls — with no upper limit. That is exactly the property that makes it fragile to outliers (Step 2).

Mini-example — this operation in isolation

Why divide by n and not by 1? The mean keeps the loss scale-free in the number of samples, so a batch of 4 and a batch of 400 with the same typical error report the *same* loss. Sum-reduction instead would make the gradient (and hence the effective learning rate) silently scale with batch size — a classic source of “my loss exploded when I increased the batch” bugs.

3 Step 2 — Huber / Smooth-L1 loss

MSE squares everything, so a single residual of 100 contributes 10 000 and hijacks the gradient. **Huber loss** (PyTorch’s `HuberLoss`; `SmoothL1Loss` is the same curve) fixes this by being quadratic near zero and *linear* once the residual passes a threshold δ . Per element, with $\delta = 1$:

$$\ell_\delta(e) = \begin{cases} \frac{1}{2} e^2, & |e| \leq \delta, \\ \delta(|e| - \frac{1}{2}\delta), & |e| > \delta, \end{cases} \quad \frac{d\ell_\delta}{de} = \text{clip}(e, -\delta, \delta).$$

Worked example on $e = [-3, -0.5, 2]$, $\delta = 1$. Classify each residual by $|e|$ versus δ , then apply the matching branch:

$$\begin{aligned} e = -3 : |e| = 3 > 1 &\Rightarrow 1 \cdot (3 - 0.5) = 2.5, \\ e = -0.5 : |e| = 0.5 \leq 1 &\Rightarrow \frac{1}{2}(0.5)^2 = 0.125, \\ e = 2 : |e| = 2 > 1 &\Rightarrow 1 \cdot (2 - 0.5) = 1.5. \end{aligned}$$

With mean reduction (PyTorch's default),

$$L_{\text{Huber}} = \frac{2.5 + 0.125 + 1.5}{3} = \frac{4.125}{3} = \boxed{1.375}.$$

The gradient simply *clips* the residual to $[-\delta, \delta]$:

$$\frac{d\ell}{de} = \text{clip}([-3, -0.5, 2], -1, 1) = [-1.0, -0.5, 1.0].$$

Compare with MSE, whose gradient $2e$ would have been $[-6, -1, 4]$ here: Huber caps the pull of the two outliers at ± 1 while still treating the small residual -0.5 exactly like MSE. That cap is the whole point.

Watch out

The two branches are deliberately stitched to be continuous and smooth at $|e| = \delta$: at $e = \delta$ both pieces give $\frac{1}{2}\delta^2$ and both derivatives give δ . Drop the $-\frac{1}{2}\delta$ term from the linear branch and the curve jumps — a discontinuous loss produces a discontinuous gradient and unstable training. Keep the $-\frac{1}{2}\delta$.

When this is the right choice

Reach for **Huber** (or Smooth-L1) when targets are real-world and noisy — sensor readings, prices, bounding-box coordinates — where a few corrupt labels would otherwise dominate MSE. Concretely, append one outlier (pred 100, target 1, so $e = 99$) to the regression batch above. MSE leaps to 1960.430 while Huber rises only to 19.815: a $\sim 99\times$ gap, because MSE squared the 99 but Huber kept it linear. Stay with plain **MSE** only when the targets are clean and you genuinely want large errors punished hardest.

4 Step 3 — Binary cross-entropy with logits

For a *binary* label $y \in \{0, 1\}$, the model outputs a raw logit z . We first squash it to a probability with the sigmoid, then score it with binary cross-entropy:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad L_{\text{BCE}} = -[y \log p + (1 - y) \log(1 - p)].$$

Worked example $z = 0.8$, $y = 1$. The sigmoid:

$$p = \frac{1}{1 + e^{-0.8}} = \frac{1}{1 + 0.449} = \frac{1}{1.449} = 0.690.$$

Since $y = 1$ only the first term survives:

$$L_{\text{BCE}} = -\log(0.690) = \boxed{0.371}.$$

The numerically stable form. Computing $\sigma(z)$ first and *then* log overflows for large $|z|$: e^{-z} can blow up, and $\log(0) = -\infty$. Algebra collapses the two steps into one overflow-free expression that PyTorch's `BCEWithLogitsLoss` uses internally:

$$L_{\text{BCE}} = \max(z, 0) - z y + \log(1 + e^{-|z|}).$$

On our numbers, $\max(0.8, 0) - 0.8 \cdot 1 + \log(1 + e^{-0.8}) = 0.8 - 0.8 + \log(1.449) = 0 + 0.371 = 0.371$ — identical to the naive route, but safe at any scale. The gradient with respect to the logit is famously clean:

$$\frac{\partial L_{\text{BCE}}}{\partial z} = p - y = 0.690 - 1 = -0.310.$$

Watch out

Use `nn.BCEWithLogitsLoss` on *raw logits* — do **not** apply `sigmoid` yourself and then `nn.BCELoss`. The split version recomputes the unstable $\log(\sigma(z))$ and can produce NaN for confident predictions; the fused version uses the $\max(z, 0)$ form above. Also note targets here are *floats* (0.0 or 1.0), not class indices.

5 Step 4 — Cross-entropy = LogSoftmax + NLL

For *multi-class* problems the model emits a vector of logits, one per class. `CrossEntropyLoss` is the composition `LogSoftmax` $\rightarrow$ `NLLLoss`: softmax turns logits into a probability distribution, and the loss is the negative log of the probability assigned to the true class.

Worked example: logits [2.0, 1.0, 0.1], **true class** 0. First softmax. Exponentiate and normalise:

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.1} = 1.105, \quad \sum = 11.213.$$

$$\text{softmax} = \left[\frac{7.389}{11.213}, \frac{2.718}{11.213}, \frac{1.105}{11.213} \right] = [0.659, 0.242, 0.099].$$

The loss is the negative log-probability of the target class (0):

$$L_{\text{CE}} = -\log(\text{softmax}_0) = -\log(0.659) = \boxed{0.417}.$$

The key gradient. Differentiating cross-entropy through the softmax produces one of the cleanest results in deep learning — the predicted distribution minus the one-hot target:

$$\frac{\partial L_{\text{CE}}}{\partial \text{logits}} = \text{softmax} - \text{onehot}.$$

With `onehot` = [1, 0, 0] for true class 0:

$$\frac{\partial L}{\partial \text{logits}} = [0.659, 0.242, 0.099] - [1, 0, 0] = [-0.341, 0.242, 0.099].$$

Read it directly: the true-class logit gets a *negative* gradient (so the optimiser pushes it *up*), and every wrong-class logit gets a positive gradient (pushed *down*), each in proportion to how much probability it wrongly claimed. The components sum to 0, as they must — softmax probabilities and a one-hot vector both sum to 1.

Watch out

Do **not** apply `softmax` before `nn.CrossEntropyLoss`. It runs `LogSoftmax` internally on *raw logits*; feeding it probabilities applies `softmax` twice and silently degrades training. Equally, the target must be an `int64` class index (0, 1, ...), *not* a one-hot float vector. These two mistakes are the most common cross-entropy bugs in the whole course.

When this is the right choice

Match the loss to the label shape. **One target value per example, continuous** $\Rightarrow$ regression: MSE (clean) or Huber (noisy). **One target, two classes** $\Rightarrow$ `BCEWithLogitsLoss` on a single logit. **One target, $C > 2$ mutually exclusive classes** $\Rightarrow$ `CrossEntropyLoss` on a C -vector of logits with an integer target. Pick by the question the labels answer, not by habit.

6 The four losses at a glance

Every number in this table is computed in the worked sections above and reproduced by the appendix code.

Loss	Input here	Scalar L	Gradient	PyTorch class
MSE	$e = [-0.5, 0.5, 0.1, 0.8]$	0.287	$2e/n$	<code>nn.MSELoss</code>
Huber	$e = [-3, -0.5, 2], \delta=1$	1.375	$\text{clip}(e, \pm\delta)$	<code>nn.HuberLoss</code>
BCE	$z = 0.8, y = 1$	0.371	$p - y = -0.310$	<code>nn.BCEWithLogitsLoss</code>
CE	$[2.0, 1.0, 0.1], t=0$	0.417	<code>softmax-onehot</code>	<code>nn.CrossEntropyLoss</code>

The gradient column is the punch line: MSE pulls in proportion to the error, Huber *caps* that pull, BCE and CE both reduce to (prediction – target) in probability space. Once you see that classification gradients are just “predicted minus true,” the backward pass stops being mysterious.

7 Equation cheat-sheet

MSE	$L = \frac{1}{n} \sum (\hat{y}_i - y_i)^2, \quad \partial_{\hat{y}_i} L = \frac{2}{n} (\hat{y}_i - y_i)$
Huber core	$ e \leq \delta : \ell = \frac{1}{2} e^2$
Huber tail	$ e > \delta : \ell = \delta(e - \frac{1}{2}\delta)$
Huber grad	$\partial_e \ell = \text{clip}(e, -\delta, \delta)$
Sigmoid	$p = \sigma(z) = 1/(1 + e^{-z})$
BCE	$L = -[y \log p + (1 - y) \log(1 - p)]$
BCE stable	$L = \max(z, 0) - zy + \log(1 + e^{- z }), \quad \partial_z L = p - y$
Softmax	$\text{softmax}_k = e^{z_k} / \sum_j e^{z_j}$
Cross-entropy	$L = -\log(\text{softmax}_t)$
CE grad	$\partial_{\text{logits}} L = \text{softmax} - \text{onehot}$

Appendix A Reference implementation

Every scalar and gradient above is reproduced by the program below, and each is checked against the matching `torch.nn` loss. All printed values round to the numbers used in the text.

```
import numpy as np
import torch, torch.nn as nn

def r(x): return round(float(x), 3)

# (1) MSE -----
yhat = np.array([2.5, 0.0, 2.1, 7.8]); y = np.array([3.0, -0.5, 2.0, 7.0])
e = yhat - y # [-0.5, 0.5, 0.1, 0.8]
mse = (e**2).mean() # 0.287
grad_mse = 2*e/len(e) # [-0.25, 0.25, 0.05, 0.4]
print("MSE", r(mse), [r(v) for v in grad_mse])
print("torch", r(nn.MSELoss()(torch.tensor(yhat), torch.tensor(y))))

# (2) Huber, delta=1, on residuals e = [-3, -0.5, 2] -----
d = 1.0; ev = np.array([-3.0, -0.5, 2.0])
def huber(e, d):
    a = abs(e)
    return 0.5*e**2 if a <= d else d*(a - 0.5*d)
hv = np.array([huber(v, d) for v in ev]) # [2.5, 0.125, 1.5]
print("Huber", r(hv.mean()), [r(v) for v in np.clip(ev, -d, d)]) # 1.375
yh = torch.tensor([0.,0.,0.]); yt = torch.tensor([3.,0.5,-2.]) # yh-yt = ev
print("torch", r(nn.HuberLoss(delta=1.0)(yh, yt)))

# (3) BCE with logits, z=0.8, y=1 -----
z, yb = 0.8, 1.0
p = 1/(1+np.exp(-z)) # 0.690
naive = -(yb*np.log(p) + (1-yb)*np.log(1-p)) # 0.371
stable = max(z,0) - z*yb + np.log(1+np.exp(-abs(z))) # 0.371
print("BCE", r(naive), r(stable), "grad", r(p-yb)) # grad p-y = -0.310
print("torch", r(nn.BCEWithLogitsLoss()(torch.tensor([z]), torch.tensor([yb]))))

# (4) Cross-entropy, logits [2,1,0.1], target class 0 -----
logits = np.array([2.0, 1.0, 0.1]); t = 0
ex = np.exp(logits - logits.max())
softmax = ex/ex.sum() # [0.659, 0.242, 0.099]
ce = -np.log(softmax[t]) # 0.417
onehot = np.array([1.,0.,0.])
grad_ce = softmax - onehot # [-0.341, 0.242, 0.099]
print("CE", r(ce), [r(v) for v in grad_ce])
print("torch", r(nn.CrossEntropyLoss()(torch.tensor(logits).unsqueeze(0),
                                     torch.tensor([t]))))

# MSE vs Huber under one outlier (e = 99) -----
yo = np.append(yhat, 100.0); to = np.append(y, 1.0); eo = yo - to
print("outlier MSE", r((eo**2).mean()),
      "Huber", r(np.array([huber(v,1.0) for v in eo]).mean())) # 1960.430 vs 19.815
```

— end of document —